Gestion d'identité et contrôle d'accès décentralisé (D-IAM)

Cours de master – version étendue

M. Amine Bouchiha

La Rochelle Université

28 octobre 2026

Plan du cours

1. Fondations : identité, IAM, menaces
2. Fondations cryptographiques
3. Identifiants décentralisés (DID)
4. Verifiable Credentials (VC)
5. Architecture SSI
6. Techniques de protection de la vie privée
7. Intégration de la blockchain
8. Contrôle d'accès décentralisé
9. Hyperledger Besu
10. Étude de cas et exercices

Fil rouge

Qui prouve quoi, à qui, avec quelles garanties, et qui doit faire confiance à qui ?

Identité numérique, IAM et modèles de menaces

Identité numérique : définitions

- **Identité numérique** : ensemble d'attributs relatifs à une personne, une organisation ou un objet, existant en ligne (identifiants, attributs, historique de transactions, traces de navigation).
- Distinguer :
  - **Identifiant** : désigne un sujet (ex. e-mail, numéro).
  - **Attribut / claim** : affirmation sur un sujet (âge, rôle, diplôme).
  - **Authentificateur** : preuve de contrôle (mot de passe, clé, biométrie).
- Le lien entre identifiant et personne réelle (*identity proofing*) est une opération distincte de l'authentification.

Fonctions d'un système IAM

Fonction	**Question posée**
Identification	Qui prétend agir ?
Authentification	Cette entité contrôle-t-elle bien l'identifiant revendiqué ?
Autorisation	Cette entité a-t-elle le droit d'effectuer cette action sur cette ressource ?
Imputabilité (*accountability*)	Peut-on tracer et auditer l'action ?
Cycle de vie	Création, mise à jour, révocation des identités et droits

Le contrôle d'accès vient *après* l'authentification : authentification $\neq$ autorisation.

Modèle de menaces : acteurs et surfaces d'attaque

Acteur / menace	**Exemples**
Attaquant externe	Hameçonnage, vol d'identifiants, *credential stuffing*, rejeu
Fournisseur d'identité compromis ou malveillant	Fuite massive, émission de fausses assertions, surveillance
Vérifieur malveillant	Sur-collecte d'attributs, revente, corrélation entre services
Utilisateur malveillant	Partage ou revente de credentials, collusion
Réseau / infrastructure	Déni de service, écoute, rupture de disponibilité du registre
Vie privée	Corrélation, traçage, déduction d'attributs

IAM centralisé

- Identité gérée par une organisation unique (université, Google, Meta...).
- **Points forts** : simplicité, administration unifiée, recours clair.
- **Limites** :
  - point de défaillance unique (disponibilité, intégrité) ;
  - base d'identifiants = cible à forte valeur (violations de données) ;
  - dépendance vis-à-vis d'un tiers, silos non interopérables ;
  - l'utilisateur ne contrôle pas ses données.

IAM fédéré : SSO, SAML, OAuth 2.0, OpenID Connect

- **SAML 2.0** : assertions XML, courant en entreprise et académie.
- **OAuth 2.0** : *délégation d'autorisation* (jetons d'accès) ; ce n'est pas un protocole d'authentification.
- **OpenID Connect** : couche d'authentification sur OAuth 2.0 (ID Token signé).

Risques résiduels : le fournisseur d'identité (IdP) voit chaque connexion (traçage), reste un point de confiance unique, et le vol de session ou d'identifiants peut affecter plusieurs services.

Pourquoi des identités décentralisées ?

Problèmes

- Violations de données, usurpation
- Collecte massive, perte de contrôle
- Contraintes RGPD (minimisation, effacement, finalité)

Promesses de la SSI

- Contrôle par le détenteur
- Moins de points centraux
- Portabilité, interopérabilité
- Divulgation minimale

Question critique

Avons-nous besoin d'une DLT ? Pas systématiquement (module 7). La SSI est avant tout un modèle de données, de confiance et de protocoles.

Clés, signatures, hachage, rotation et recouvrement

Chiffrement vs signature : deux objectifs distincts

	Chiffrement asymétrique	**Signature numérique**
Objectif	Confidentialité	Authenticité, intégrité, non-répudiation (sous hypothèses)
Opération émetteur	chiffre avec PK_B	signe avec SK_A
Opération récepteur	déchiffre avec SK_B	vérifie avec PK_A
Qui détient quoi	seul B peut lire	seul A peut produire la signature

Note importante

On ne « déchiffre » jamais avec la clé publique de B : $\mathrm{decrypt}(c, SK_B)$. La clé publique ne sert qu'à chiffrer ou vérifier.

Ce que prouve (et ne prouve pas) une signature

- $\mathrm{Verify}(PK, m, \sigma) = 1$ prouve que **le détenteur de la clé privée associée à** PK a signé m, sous les hypothèses : sécurité du schéma (EUF-CMA), clé non compromise, vérificateur correct.
- Elle **ne prouve pas** :
  - l'identité réelle du signataire ;
  - que la clé n'a pas été volée ou copiée ;
  - la véracité du contenu de m.
- Lier PK à une personne/organisation exige un tiers de confiance, une procédure de vérification (identity proofing) ou un credential.

Hachage et engagements

- Fonction de hachage H : résistance aux préimages, secondes préimages et collisions (ex. SHA-256, SHA-3).
- Usages en D-IAM : intégrité d'un document (ancrage d'un hash), arbres de Merkle, *commitments*.
- **Engagement** avec sel : $c = H(m \,\|\, r)$, r aléatoire secret.
  - *hiding* : c ne révèle pas m ;
  - *binding* : impossible d'ouvrir c vers $m' \neq m$.
- **Piège** : hacher une donnée à faible entropie (date de naissance) sans sel est réversible par énumération.

Courbes, formats et algorithmes courants

Algorithme	**Usage typique**	**Remarque**
Ed25519 (EdDSA)	did :key, signatures JOSE/COSE	rapide, déterministe
ECDSA secp256k1	écosystème Ethereum, did :ethr	signature non déterministe sans RFC 6979
ECDSA P-256	wallets mobiles, secure enclave	compatibilité matérielle
BBS+ / BBS	divulgation sélective, non-chaînabilité	signatures multi-messages

Les clés sont décrites dans le DID document par des *verification methods* (JWK ou multibase).

Rotation, compromission et recouvrement

- **Rotation planifiée** : nouvelle clé publiée, ancienne désactivée après une période de transition.
- **Compromission** : révoquer la clé dans le DID document, ré-émettre les credentials liés, auditer les signatures émises depuis la date supposée du vol.
- **Recouvrement** (*recovery*) :
  - sauvegarde chiffrée de la seed ;
  - *social recovery* (seuil *k*-sur-*n* de tiers de confiance) ;
  - clés de recouvrement pré-engagées (ex. *pre-rotation*, KERI) ;
  - multi-signature / MPC pour les entités.
- **Tension** : plus la récupération est facile, plus la surface d'attaque augmente.

DID Core, méthodes, documents et résolution

Qu'est-ce qu'un DID ? (W3C DID Core 1.0)

- Identifiant globalement unique, **résoluble** vers un DID document, sous le contrôle d'un *controller*.
- Syntaxe : `did:<méthode>:<identifiant-spécifique>`

```
did:example:123456789abcdefghi
did:web:example.edu
did:key:z6Mk... did:ethr:0xb9c5714089478a327f09197987f16f9e5d936e8a
```

Note importante

Un DID n'est **pas nécessairement une adresse blockchain**. Il peut être dérivé d'une clé (did :key), d'un domaine (did :web), d'un registre distribué ou d'un autre système. Un DID peut identifier une personne, une organisation ou une chose.

Méthodes DID : comparaison

Méthode	Ancrage de confiance	Atouts	Limites
did :key	La clé elle-même	Sans infrastructure, éphémère	Pas de rotation ni de mise à jour
did :web	DNS + TLS	Simple, adapté aux organisations	Dépend du domaine et de l'hébergeur
did :ethr	Compte Ethereum + registre ERC-1056	Rotation de clés, délégations	Coûts, lien visible entre DID et adresse
did :ion / did :peer	Réseau ancré / échange pair à pair	Passage à l'échelle, vie privée	Complexité, maturité variable

Le choix de la méthode détermine les garanties de résolution, de mise à jour et de désactivation.

Le DID document

```
{
 "@context": ["https://www.w3.org/ns/did/v1"],
 "id": "did:web:example.edu",
 "controller": "did:web:example.edu",
 "verificationMethod": [{
   "id": "did:web:example.edu#key-1",
   "type": "JsonWebKey",
   "controller": "did:web:example.edu",
   "publicKeyJwk": {"kty":"OKP","crv":"Ed25519","x":"..."} }],
 "authentication": ["did:web:example.edu#key-1"],
 "assertionMethod": ["did:web:example.edu#key-1"],
 "service": [{"id":"#oid4vci","type":"OpenID4VCI",
   "serviceEndpoint":"https://example.edu/issuer"}]
}
```

Relations de vérification : `authentication`, `assertionMethod`, `keyAgreement`, `capabilityInvocation`, `capabilityDelegation`.

Ce que contient (et ne contient pas) un DID document

Contient

- Clés publiques et leurs usages autorisés
- Endpoints de service
- Contrôleur(s)

Ne contient pas / ne prouve pas

- Le nom ou l'identité civile du sujet
- Que le sujet est une personne réelle donnée
- Le niveau de confiance accordable

Principe

Le DID prouve le *contrôle* d'un identifiant. L'*attribution à une identité réelle* repose sur des credentials, des procédures de vérification et des relations de confiance.

Règle de vie privée : ne jamais placer de données personnelles dans un DID document public.

Résolution d'un DID : exemple détaillé

1. **Entrée** : `did:web:example.edu:users:alice`
2. Parsing : méthode `web`, identifiant `example.edu:users:alice`.
3. Règle de la méthode did :web : les `:` deviennent `/` :

   ```
   https://example.edu/users/alice/did.json
   ```

   (sans chemin : `https://example.edu/.well-known/did.json`)
4. Requête HTTPS, validation TLS, récupération du JSON.
5. **Contrôles** : `id` du document = DID demandé, syntaxe conforme, clé de vérification présente pour la relation attendue.
6. **Sortie** : DID document + métadonnées de résolution (dates, désactivation).

Sécurité : la confiance repose ici sur DNS/TLS et sur le contrôle du serveur web.

Authentification par DID et protection contre le rejeu

Défi-réponse entre vérifieur V et détenteur H :

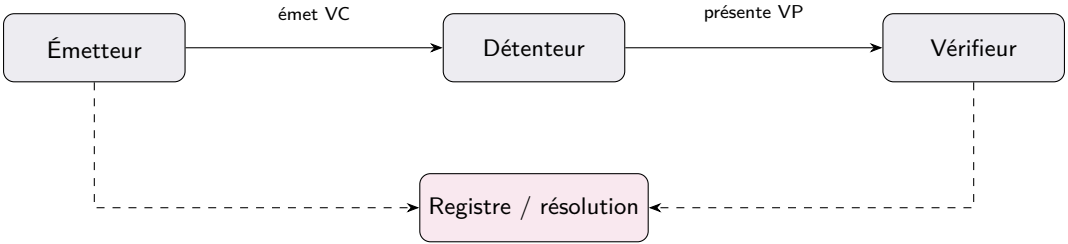
1. $V \to H$: nonce n aléatoire, domaine d ; V conserve $(n, \text{expiration})$.
2. $H \to V$: $\sigma = \mathrm{Sign}_{sK}(H(n \,\|\, d \,\|\, \mathrm{DID}_V))$ et son DID.
3. V : résout le DID, vérifie σ avec une clé de la relation `authentication`, vérifie que n est frais et non consommé, que d correspond.

Attaques et parades

- **Rejeu** : nonce à usage unique + expiration courte.
- **Relais / MITM** : lier la signature au domaine/audience et au canal.
- **Clé volée** : stockage matériel, rotation, révocation.

Émission, présentation, vérification et révocation

Modèle de données VC (W3C VC Data Model 2.0)



VC : ensemble d'assertions sur un sujet, signé par l'émetteur. **VP** : présentation, signée par le détenteur, liée à un défi (nonce, audience).

Exemple de credential (diplôme)

```
{
 "@context": ["https://www.w3.org/ns/credentials/v2"],
 "type": ["VerifiableCredential","UniversityDegreeCredential"],
 "issuer": "did:web:example.edu",
 "validFrom": "2026-06-30T00:00:00Z",
 "credentialSubject": {
   "id": "did:key:z6MkAlice...",
   "degree": {"type":"MasterDegree","name":"Cybersecurite"} },
 "credentialStatus": {
   "type": "BitstringStatusListEntry",
   "statusPurpose": "revocation",
   "statusListIndex": "94567",
   "statusListCredential": "https://example.edu/status/3" },
 "proof": { "type":"DataIntegrityProof", "cryptosuite":"eddsa-rdfc-2022", ... }
}
```

Formats de sécurisation

Format	Principe	Propriétés
JWT-VC (JOSE)	Signature JWS du credential	Large support, divulgation tout-ou-rien
Data Integrity	Preuve intégrée, canonisation RDF	Cryptosuites variées (EdDSA, ECDSA, BBS)
SD-JWT VC	Hachages salés des claims, divulgation sélective	Simple, liant l'émetteur ; chaînabilité si réutilisé
ISO mdoc (18013-5)	CBOR/COSE, éléments individuels	Permis mobile, EUDI Wallet
BBS / AnonCreds	Preuves à divulgation nulle, non-chaînabilité	Prédicats, mais écosystème moins uniforme

Protocoles OpenID pour les credentials

- **OpenID4VCI** : émission (flux *pre-authorized code* ou *authorization code*, endpoint de credential, preuve de possession de la clé du détenteur).
- **OpenID4VP** : présentation (requête d'autorisation avec *presentation definition* ou DCQL, nonce, audience).
- **SIOPv2** : le détenteur agit comme fournisseur OpenID auto-émis (ID token signé par sa clé).
- **HAIP** : profil à haute assurance (eIDAS 2.0 / EUDI Wallet).
- **Alternatives** : DIDComm v2 (messagerie pair à pair authentifiée), WACI, Hyperledger Aries.

Cycle de vie : émission, stockage, présentation

1. **Émission** : l'émetteur vérifie les attributs, lie le credential à une clé/DID du détenteur (*holder binding*), signe.
2. **Stockage** : dans le wallet du détenteur, chiffré, sauvegardé ; les données restent hors chaîne.
3. **Requête** : le vérifieur indique les attributs nécessaires et la finalité.
4. **Présentation** : le détenteur compose une VP (divulgation limitée), la signe avec nonce et audience.
5. **Vérification** : voir diapositive suivante.
6. **Décision** : application de la politique d'accès.

Vérification d'une présentation : étapes

1. Signature de la VP valide, nonce frais, audience correcte.
2. **Holder binding** : le sujet du VC correspond au signataire de la VP.
3. Résolution du DID de l'émetteur, signature du VC valide sous une clé `assertionMethod` *valable à la date d'émission*.
4. Dates de validité (`validFrom`, `validUntil`).
5. **Statut** : credential non révoqué/suspendu.
6. **Confiance dans l'émetteur** : l'émetteur figure-t-il dans la liste/registre de confiance pour ce type de credential ?
7. Conformité au schéma et à la politique du vérifieur.

Point clé

Une signature valide n'implique pas que l'émetteur est *autorisé* à attester ce fait : l'étape 6 est indispensable.

Révocation : Bitstring Status List

- Le credential porte un index i dans une liste publiée par l'émetteur sous forme de bitstring compressé (GZIP, base64).
- Le vérifieur télécharge la liste et lit le bit i : $1 \Rightarrow$ révoqué (ou suspendu).
- **Vie privée** : liste large (minimum recommandé 131 072 entrées) pour une confusion statistique ; le vérifieur ne révèle pas l'index à l'émetteur s'il met la liste en cache.
- **Compromis** : fraîcheur (TTL du cache) vs charge et traçage ; disponibilité de l'endpoint.
- Alternatives : accumulateurs cryptographiques, credentials de courte durée (réémission), registre on-chain (coût, visibilité).

Registres de confiance et gouvernance des émetteurs

- Question : *qui est autorisé à émettre quel type de credential ?*
- Mécanismes : listes de confiance (ex. EU Trusted Lists), registres fédérés, chaînes de certification X.509, *trust frameworks*, entités d'accréditation.
- Peuvent être hébergés hors chaîne, signés, ou ancrés sur un registre distribué.
- Un registre définit : périmètre, critères d'admission, procédure de retrait, audit, responsabilités.
- **Risque** : recentralisation de fait autour de l'autorité du registre.

Modèle émetteur–détenteur–vérifieur, wallets, gouvernance

Modèle Issuer–Holder–Verifier

Rôle	Responsabilités	Menaces principales
Émetteur	Vérifier les attributs, signer, gérer les statuts	Compromission de clé, émission frauduleuse
Détenteur	Garder les clés, choisir quoi divulguer	Vol de wallet, hameçonnage, partage
Vérifieur	Définir politique, vérifier, minimiser les données	Sur-collecte, corrélation, mauvaise vérification
Registre	Publier clés, statuts, listes de confiance	Indisponibilité, falsification, censure

Wallets d'identité

- **Fonctions** : génération/gestion de clés, stockage de credentials, consentement, présentations, sauvegarde.
- **Types** : mobile (secure enclave/StrongBox), cloud (custodial), matériel, wallet d'organisation.
- **Enjeux** :
  - attestation du wallet (le vérifieur peut-il exiger un wallet certifié ?) ;
  - protection contre la copie et le partage (liaison matérielle) ;
  - ergonomie du consentement ;
  - recouvrement en cas de perte (module 2).
- **Tension** : exiger l'attestation du wallet réduit la fraude mais peut exclure des wallets légitimes.

Relations de confiance et interopérabilité

- **Confiance transitive limitée** : on ne fait pas confiance à un DID, on fait confiance à une *autorité pour une assertion donnée*.
- **Couches d'interopérabilité** :
  - identifiants (DID Core, méthodes, résolveur universel) ;
  - format de credential (JWT, SD-JWT, mdoc, Data Integrity) ;
  - protocoles de transport (OpenID4VCI/VP, DIDComm) ;
  - vocabulaires et schémas sémantiques.
- Sans accord sur chaque couche, l'interopérabilité est partielle : d'où les *profils*.
- Organismes : W3C, DIF, IETF, OpenID Foundation, ISO/IEC, ETSI, ToIP, Hyperledger.

Gouvernance et responsabilité

- Qui définit les règles : fonctionnement du réseau, admission des émetteurs, résolution des litiges ?
- Modèles : consortium, gouvernement (eIDAS 2.0), fondation, communauté ouverte.
- **Cadre juridique** : valeur légale des attestations, responsabilité de l'émetteur, RGPD, eIDAS.
- **Critères de conception** : transparence, auditabilité, voies de recours, inclusion (personnes sans smartphone).

Pairwise DIDs, divulgation sélective, non-chaînabilité, ZKP

Risques de corrélation

- **Corrélation par identifiant** : même DID réutilisé chez deux vérifieurs.
- **Corrélation par signature/credential** : même signature d'émetteur ou même valeur présentée plusieurs fois.
- **Corrélation par collusion** : émetteur et vérifieur comparent leurs journaux (*phone home* via la révocation).
- **Corrélation par métadonnées** : adresse IP, horodatage, service endpoint.
- **Corrélation sur chaîne publique** : toute adresse réutilisée est traçable.

Pairwise DIDs

- Un DID distinct **par relation** (did :peer, did :key éphémère).
- Le détenteur évite que deux vérifieurs relient ses activités via l'identifiant.
- **Limites** :
  - inutile si le credential contient lui-même un identifiant stable (numéro étudiant) ;
  - lourdeur de gestion ; clés à sauvegarder par relation ;
  - la signature de l'émetteur reste un identifiant unique du credential.
- Nuance par rapport aux slides initiaux : « plusieurs DIDs » réduit le traçage mais ne l'élimine pas.

Divulgation sélective et non-chaînabilité

	SD-JWT	**BBS**	**ZKP (AnonCreds, zk-SNARK)**
Divulgation sélective	Oui (claims individuels)	Oui	Oui
Prédicats (age $\geq$ 18)	Non (sauf claim dédié)	Limité	Oui
Non-chaînabilité	Non (signature fixe, mitigée par lots)	Oui	Oui
Coût	Faible	Moyen	Élevé

Minimisation : ne demander que l'attribut nécessaire (“majeur” et non la date de naissance).

Preuves à divulgation nulle (ZKP) : principe

- Prouver une affirmation $\exists w : R(x, w) = 1$ sans révéler w.
- Propriétés : **complétude**, **solidité**, **zero-knowledge**.
- Familles : Σ-protocoles non interactifs (Fiat–Shamir), zk-SNARK (Groth16, PLONK), zk-STARK.
- Applications D-IAM :
  - preuve de possession d'un credential valide sans le montrer ;
  - prédicats (seuil d'âge, appartenance à un ensemble) ;
  - preuve de non-révocation.
- **Hypothèses** : setup de confiance (Groth16), sécurité du circuit, liaison au nonce du vérifieur (sinon rejeu).

Exemple : prouver ≥ 18 ans sans révéler la date de naissance

Données : le credential signé par l'émetteur contient dob (jours depuis l'epoch).
Énoncé public $x = (PK_{\text{émetteur}},\ T,\ n)$ avec $T =$ aujourd'hui $- 18$ ans, nonce n.
Témoin secret $w = (\mathrm{dob},\ \sigma,\ r)$.
Relation $R(x, w) = 1$ si et seulement si :

1. $\mathrm{Verify}(PK_{\text{émetteur}}, \mathrm{dob}, \sigma) = 1$ (le credential est authentique) ;
2. $\mathrm{dob} \leq T$ (preuve d'intervalle) ;
3. la preuve est liée à n (contre le rejeu).

Le vérifieur apprend : « le détenteur d'un credential valide de cet émetteur a au moins 18 ans ». Il n'apprend ni dob ni σ.
Variante pragmatique : l'émetteur signe directement un claim booléen `age_over_18` (SD-JWT), plus simple mais sans preuve de prédicat.

Séparation on-chain/off-chain, coûts, immutabilité et RGPD

Faut-il une blockchain ?

Utile quand...	**Superflue quand...**
Plusieurs organisations sans autorité commune doivent partager un registre	Une autorité unique et fiable existe déjà
Besoin d'auditabilité et d'horodatage résistants à la falsification	Performances et latence critiques
Résolution de DID/clés sans dépendre d'un opérateur	Données volatiles ou confidentielles

Principe

La blockchain apporte un *ordre partagé et une intégrité de registre*, pas la vérité des attributs. did :web ou did :key suffisent dans de nombreux cas.

Que mettre sur la chaîne, que garder hors chaîne

On-chain (minimal)

- Ancrage de clés/DID documents (ou leur hash)
- Événements de rotation/désactivation
- Registres d'émetteurs accrédités
- Racines de listes de statut (hash)

Off-chain

- Credentials et attributs personnels
- Présentations
- Documents volumineux
- Données chiffrées éventuelles

Règle

Aucune donnée personnelle, même chiffrée ou hachée à faible entropie, dans un registre immuable.

Coûts, immutabilité et RGPD

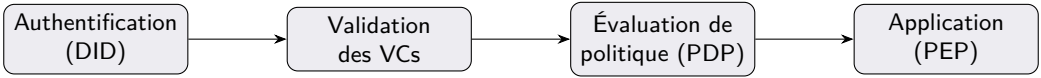
- **Coûts** : frais de gas par écriture, latence de finalité, passage à l'échelle ; privilégier L2, réseaux permissionnés, ancrage par lots (Merkle).
- **Immutabilité vs droit à l'effacement (art. 17)** : une donnée écrite ne peut être supprimée.
- **Position défendable** : stocker hors chaîne, ne publier que des engagements non réversibles ; supprimer la donnée hors chaîne rend l'engagement inexploitable.
- **Qualification des hachages** : un hash peut rester une donnée personnelle s'il est ré-identifiable.
- **Responsable de traitement** : à identifier dans un réseau distribué (consortium, opérateurs de noeuds).
- Consulter les lignes directrices EDPB sur la blockchain.

Menaces propres à l'intégration blockchain

Menace	Mitigation
Traçage d'adresses	DIDs par relation, pas de PII, mixage de relations
Censure / réorganisation	Réseau avec finalité, consortium, plusieurs sources de résolution
Clé de contrôleur perdue	Recouvrement, multisig, délégation
Oracle compromis	Preuves signées, multiples oracles, vérification locale
Déni de service	Limiter les écritures, mise en cache des documents, résolution hors chaîne
Coût d'écriture élevé	Ancrage par lots, L2

RBAC, ABAC, évaluation de politique, application et révocation

Du DID au contrôle d'accès



- **PEP** (Policy Enforcement Point) : intercepte la requête.
- **PDP** (Policy Decision Point) : évalue la politique, renvoie Permit/Deny.
- **PAP / PIP** : administration des politiques / sources d'attributs.

Modèles : RBAC, règles, ABAC

Modèle	Principe	Limites
RBAC	Permissions attachées à des rôles ; l'utilisateur obtient des rôles (VC `role`)	Explosion de rôles, peu contextuel
Basé règles	Conditions fixes (horaires, VPN)	Rigide, difficile à composer
ABAC	Décision selon attributs sujet, ressource, action, contexte	Politiques complexes, vérification de la provenance des attributs

Dans un cadre décentralisé, chaque attribut doit avoir une **source de confiance** (émetteur accrédité), pas seulement une valeur.

Exemple d'évaluation de politique ABAC

Politique : « Un employé RH peut lire les fiches salariales depuis un appareil autorisé, en heures ouvrées. »

```
permit if
  subject.vc.type == "EmployeeCredential"
  and subject.vc.department == "RH"
  and issuer(subject.vc) in TrustedIssuers["Employee"]
  and not revoked(subject.vc)
  and action == "read" and resource.type == "payslip"
  and context.device in AuthorizedDevices
  and context.time in [08:00, 18:00]
default deny // deny-overrides
```

Requête : Alice (RH, appareil A12, 10h30) $\rightarrow$ read payslip $\Rightarrow$ **Permit**.
Même requête à 21h, ou avec un credential révoqué $\Rightarrow$ **Deny**.

Application de la décision et révocation

- **Avant l'accès** : vérifier signature, statut, issuer de confiance, nonce.
- **Pendant la session** : durée de vie courte du jeton d'accès, ré-évaluation sur changement d'attribut.
- **Après** : journal d'audit signé, sans données superflues.
- **Révocation** : consultation du statut à l'accès (fraîcheur) ou credentials de courte durée.
- **Capability-based** : jetons de capacité délégables (ZCAP-LD, UCAN), atténuation des droits à chaque délégation.
- **Défauts** : refuser par défaut, séparer PEP et PDP, valider les entrées.

Contrôle d'accès par smart contract (RBAC on-chain)

```
// Solidity, schema simplifie (OpenZeppelin AccessControl)
contract Registry is AccessControl {
  bytes32 public constant WRITER = keccak256("WRITER");
  mapping(bytes32 => bytes32) public entries;
  constructor(address admin) { _grantRole(DEFAULT_ADMIN_ROLE, admin); }
  function put(bytes32 k, bytes32 v) external onlyRole(WRITER) {
    entries[k] = v;
  }
}
```

Limites : le contrat voit des adresses, pas des credentials ; l'ABAC complexe exige des oracles ou des vérifications de preuves on-chain (coût). Les rôles et les décisions sont publics : attention à la fuite d'information.

Permissioning de noeuds et de comptes, gouvernance, sécurité opérationnelle

Pourquoi le permissioning ?

- Réseau privé/consortium : seuls des noeuds et comptes identifiés doivent participer.
- Deux niveaux :
  - **Node permissioning** : quels pairs peuvent se connecter ;
  - **Account permissioning** : quels comptes peuvent soumettre des transactions.
- Deux mises en oeuvre : **locale** (fichier) et **on-chain** (smart contract).
- Le permissioning *ne remplace pas* l'autorisation applicative des smart contracts : c'est une couche réseau.

Permissioning local (fichier de configuration)

```
# permissions_config.toml (a placer dans le dossier de donnees du noeud)
nodes-allowlist = ["enode://<pubkey>@10.0.0.2:30303"]
accounts-allowlist = ["0xfe3b5...", "0x627306..."]
```

Options de démarrage (à vérifier selon la version de Besu) :

```
besu --permissions-nodes-config-file-enabled \
     --permissions-accounts-config-file-enabled
```

- **Avantages** : simple, aucune dépendance à la chaîne.
- **Limites** : chaque noeud a sa propre liste ⇒ risque de divergence, mise à jour manuelle (propagation, rechargement), peu auditable.

Permissioning on-chain

- Des smart contracts (noeuds, comptes, admin) gèrent les listes ; tous les noeuds interrogent le même état.
- Options : `-permissions-nodes-contract-enabled`, `-permissions-accounts-contract-enabled` avec les adresses des contrats.
- **Avantages** : cohérence, mise à jour dynamique, traçabilité des changements, gouvernance programmable (multisig, vote).
- **Limites** :
  - dépendance à la sécurité des comptes administrateurs ;
  - amorçage (genesis) et risque d'auto-verrouillage ;
  - les listes sont visibles par les participants.

Gouvernance et sécurité opérationnelle

Sujet	**Pratiques recommandées**
Administration	Multisig ou vote des membres pour ajouter/retirer noeuds et comptes ; séparation des rôles
Clés des validateurs	HSM ou coffre, sauvegarde, rotation, journalisation d'accès
Consensus (QBFT/IBFT)	Seuil de tolérance f fautifs pour $3f+1$ validateurs ; surveillance des validateurs inactifs
Réseau	Isolation, TLS pour RPC, limitation de débit, pare-feu
API RPC	Désactiver les méthodes d'administration publiques, authentification
Mises à jour	Plan de mise à niveau coordonné, tests de non-régression

Credentials universitaires, vérification d'âge, politiques d'accès

Cas : diplômes universitaires vérifiables

- **Acteurs** : université (émetteur), étudiant (détenteur), employeur (vérifieur), registre d'accréditation.
- **Architecture proposée** :
  - did :web pour l'université, did :key ou pairwise pour l'étudiant ;
  - credential SD-JWT VC (diplôme, intitulé, année) ;
  - OpenID4VCI pour l'émission, OpenID4VP pour la présentation ;
  - liste de statut pour la révocation (erreur administrative, fraude) ;
  - liste de confiance des universités accréditées.
- **Aucune donnée sur chaîne** ; blockchain seulement optionnelle pour un registre d'accréditation partagé entre pays.

Cas : vérification d'âge et politique d'accès

1. L'utilisateur détient un credential d'identité (émetteur : autorité publique).
2. Le service demande uniquement la preuve ≥ 18 (finalité, nonce, audience).
3. Le wallet produit une preuve de prédicat ou un claim `age_over_18` divulgué sélectivement.
4. Le vérifieur contrôle : signature, émetteur autorisé pour ce type, statut, nonce, expiration.
5. Politique : `permit if age_over_18 and issuer` $\in$ `TrustedGov`, sinon `deny`.
6. Journal : décision + hash de transaction, **sans** date de naissance ni identifiant stable.

Compromis : la preuve ZK est plus protectrice mais plus lourde ; le claim booléen est plus simple mais chaînable.

Scénarios d'attaque et mesures

Attaque	Effet	Mesure
Vol de clé du détenteur	Usurpation	Stockage matériel, rotation, révocation du credential
Émetteur compromis	Faux diplômes	Révocation de la clé émetteur, rotation, audit des émissions, listes de confiance datées
Vérifieur malveillant	Fuite d'attributs	Minimisation, divulgation sélective, pairwise
Rejeu d'une présentation	Accès frauduleux	Nonce, audience, expiration
Corrélation	Traçage	Credentials à usage unique, non-chaînabilité
Déni de service sur résolveur ou liste de statut	Indisponibilité	Cache, redondance, politique de repli explicite

Arbitrages de conception

Axe	Option A	Option B
Méthode DID	did :web (simple, DNS)	did :ethr (décentralisé, coûts)
Divulgation	SD-JWT (simple)	BBS/ZKP (non-chaînable)
Révocation	Liste de statut (fraîcheur)	Credentials courts (réémission)
Registre de confiance	Autorité centrale	Consortium sur Besu
Récupération	Sauvegarde cloud (pratique)	Social recovery (souveraineté)

Aucune option n'est dominante : l'analyse doit relier choix techniques, modèle de menaces et exigences légales.

Exercices (1/2) : analyse

1. Un attaquant remplace le DID document d'un did :web en piratant le serveur. Quelles garanties sont perdues ? Quelles mesures de détection proposer ?
2. Montrer qu'une signature valide d'un VC ne suffit pas à accepter un diplôme. Quelles vérifications complémentaires sont nécessaires ?
3. Un vérifieur utilise un hash SHA-256 de la date de naissance publié on-chain. Décrire une attaque par énumération et estimer le nombre d'essais.
4. Comparer les risques de corrélation entre SD-JWT, BBS et ZKP lorsqu'un utilisateur présente le même credential à dix vérifieurs.
5. Un utilisateur perd son téléphone. Concevoir un protocole de recouvrement et évaluer ses risques.

Exercices (2/2) : conception d'un système d'identité décentralisé

Énoncé. Concevoir un système pour une fédération de cinq universités (étudiants, personnels, accès aux ressources numériques, vérification d'âge pour les services partenaires).
Livrables :

1. Acteurs, relations de confiance, modèle de menaces (au moins six menaces).
2. Choix des méthodes DID, formats de credentials, protocoles ; justification.
3. Ce qui est on-chain vs off-chain, et pourquoi (RGPD).
4. Politiques d'accès (RBAC + ABAC) avec exemple d'évaluation.
5. Gestion de la révocation, de la rotation et du recouvrement.
6. Gouvernance (si Besu : permissioning, administration).
7. Évaluation critique : coûts, vie privée, utilisabilité, limites.

Questions de discussion

- Une identité « auto-souveraine » est-elle compatible avec l'exigence légale de vérification d'identité (KYC) ?
- Qui devrait contrôler les registres de confiance dans un système décentralisé ?
- La non-chaînabilité est-elle compatible avec l'imputabilité et la lutte contre la fraude ?
- Faut-il privilégier la simplicité (did :web, SD-JWT) ou la robustesse cryptographique (ZKP) ?
- Un wallet certifié par un État est-il encore « souverain » ?

Synthèse

- Un **DID** prouve le contrôle d'un identifiant, pas l'identité réelle : celle-ci repose sur des credentials et des relations de confiance.
- Une **signature** prouve le contrôle d'une clé sous hypothèses ; chiffrement et signature ont des rôles distincts.
- La **blockchain** est un composant optionnel : minimale, sans données personnelles.
- La **vie privée** exige minimisation, divulgation sélective, unlinkability et prudence face à la corrélation.
- Le **contrôle d'accès** combine vérification de credentials, confiance dans l'émetteur, politique (RBAC/ABAC) et révocation.
- **Besu** : permissioning local ou on-chain, avec gouvernance et sécurité opérationnelle.

Références

- W3C, *Decentralized Identifiers (DIDs) v1.0*, Recommandation, 2022.
- W3C, *Verifiable Credentials Data Model v2.0*, *Bitstring Status List*, *Data Integrity*.
- OpenID Foundation, *OpenID for Verifiable Credential Issuance* et *OpenID for Verifiable Presentations*, *SIOPv2*, *HAIP*.
- IETF, *SD-JWT* (RFC 9901) et *SD-JWT-based Verifiable Credentials* ; RFC 7515/7519 (JOSE/JWT).
- ISO/IEC 18013-5 (mDL) ; ISO/TR 23249 (DLT et identité).
- Règlement eIDAS 2.0 et *EUDI Wallet ARF* ; RGPD ; lignes directrices EDPB sur la blockchain.
- Hyperledger Besu, documentation sur le *permissioning*.
- Allen C., *The Path to Self-Sovereign Identity*, 2016.
- Sovrin Foundation ; Hyperledger Aries, Indy, Ursa ; Decentralized Identity Foundation (DIF).